Madhav Institute of Technology & Science, Gwalior (M.P.), INDIA

Deemed University
(Declared under Distinct Category by Ministry of Education, Government of India)
NAAC ACCREDITED WITH A++ GRADE

Micro Project-II 34251210

Weather Data Analysis

A report submitted in partial fulfilment of the requirement for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND TECHNOLOGY

Submitted by:

Prerna Mishra (BTTC25O1051)

Priyal Choudhary (BTTC25O1053)

Faculty Mentor:

Mrs. Aditi Samadhiya

Assistant Professor

Submitted to:

CENTRE FOR COMPUTER SCIENCE AND TECHNOLOGY

January - June 2026

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

Deemed University
(Declared under Distinct Category by Ministry of Education, Government of India)
NAAC ACCREDITED WITH A++ GRADE

CERTIFICATE

This is to certify that **Prerna Mishra (BTTC25O1051) and Priyal Choudhary (BTTC25O1053)** have submitted the Micro Project-II report under the mentorship of Mrs. Aditi samadhiya, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in **COMPUTER SCIENCE AND TECHNOLGY** from Madhav Institute of Technology and Science, Gwalior.

Mrs. Aditi Samadhiya
Assistant Professor
Centre for CST

Dr. Abhishek Dixit
Head of Department
Centre for CST

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

Deemed University
(Declared under Distinct Category by Ministry of Education, Government of India)
NAAC ACCREDITED WITH A++ GRADE

DECLARATION

We hereby declare that the work being presented in this project report, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science And Technology at Madhav Institute of Technology & Science, Gwalior is an authenticated and original record of our work under the mentorship of **Mrs. Aditi Samadhiya, Assistant professor, Centre for Computer Science and Technology.**

We declare that we have not submitted the material embodied in this report for the award of any other degree or diploma elsewhere.

Date:
Place: Gwalior

Prerna Mishra (BTTC25O1051)
Priyal Choudhary (BTTC25O1053)
Ist Year,
Computer Science and Technology

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR

Deemed University
(Declared under Distinct Category by Ministry of Education, Government of India)
NAAC ACCREDITED WITH A++ GRADE

ACKNOWLEDGEMENT

The full semester project has proved to be pivotal to our career. We are thankful to our institute, **Madhav Institute of Technology and Science** for allowing us to undertake our disciplinary/interdisciplinary project as a curriculum requirement, under the provisions of the Flexible Curriculum Scheme (based on the AICTE Model Curriculum 2018), approved by the Academic Council of the institute. We extend our gratitude to the Vice-Chancellor of the institute, **Dr. R. K. Pandit** and Pro- Vice-Chancellor, **Dr. Manjaree Pandit** for their support.

We would sincerely like to thank, **Centre for Computer Science and Technology,** for allowing us to explore this project. We humbly thank **Dr. Abhishek Dixit, Assistant Professor and Head, Centre for Computer Science and Technology,** for his continued support during the course of this engagement, which eased the process and formalities involved.

We are sincerely thankful to our faculty mentor. We are grateful for the guidance of **Mrs. Aditi Samadhiya**, **Assistant professor, Centre for Computer Science and Technology**, for her continued support and guidance throughout the project. We are also very thankful to the faculty and staff of the centre.

Prerna Mishra (BTTC25O1051)
Priyal Choudhary (BTTC25O1053)
I Year,
Computer science and technology

PLAGIARISM CHECK CERTIFICATE

This is to certify that We, students of B. Tech. in **Centre for Computer Science and Technology,** have checked our complete report entitled **"Weather Data Analysis "** for similarity/plagiarism using the "Turnitin" software available in the institute.

This is to certify that the similarity in our report is found to be ……% with AI writing .....% which is within the specified limit (20%).
The full plagiarism report, along with the summary, is enclosed.

Prerna Mishra
(BTTC25O1051)
Priyal Choudhary
(BTTC25O0153)

Checked & Approved By:

Mrs. Aditi Samadhiya
Assistant Professor
Centre for Computer Science and Technology
MITS Deemed University, Gwalior

S.No.	Content	Page No.
1	Introduction	
2	Objective	
3	Technology stack	
4	System design	
5	Codes	
6.	Analysis of results	
7.	Conclusion	
8.	Future scope	
9.	References	

LIST OF FIGURES

FIGURE NUMBER	FIGURE CAPTION	PAGE NO.
FIG. 1	HOW THE SYSTEM WORK	
FIG. 2- 24	CODES	
FIG. 25-27	OUTPUT	

1. Introduction

Weather impacts our lives in many ways. has an impact on our lives in more ways than one. It determines our diet, our mode of transportation and even the process by which farmers grow their crops. dressing, mode of transportation, and even the process through which farmers cultivate their crops. So this requires us to know the different weather conditions. This, therefore, makes it necessary for us to have knowledge regarding various weather conditions. The truth is that a large amount of weather data is collected daily using technologies such as machines, satellites and weather stations. there are quite a number of weather data collected daily using technology such as machines, satellites, and weather stations. Analysis of weather-related data involves the interpretation of all these collected weather data. Data analysis in relation to weather involves the interpretation of all this collected weather data. During this analysis, weather components such as temperature, humidity, precipitation and wind speed are taken into account. rainfall, and wind speeds are considered. From such an analysis, we can predict the behavior of the weather. one can be able to predict the behavior of the weather. The main objective of carrying out this project is to analyze the collected meteorological data in order to extract useful information. primary goal of conducting this project is to analyze the weather data collected in order to extract useful information from it. This will make weather forecasts easier, among other things. assist in weather predictions among other things. The project will make it easier for us to appreciate the usefulness of data analysis in addressing real-world problems such as weather changes. dealing with real-world issues such as weather changes. Weather impacts our lives in many ways. has an impact on our lives in more ways than one. It determines our diet, our mode of transportation and even the process by which farmers grow their crops. dressing, mode of transportation, and even the process through which farmers cultivate their crops. So this requires us to know the different weather conditions. This, therefore, makes it necessary for us to have knowledge regarding various weather conditions. The truth is that a large amount of weather data is collected daily using technologies such as machines, satellites and weather stations. there are quite a number of weather data collected daily using technology such as machines, satellites, and weather stations. Analysis of weather-related data involves the interpretation of all these collected weather data. Data analysis in relation to weather involves the interpretation of all this collected weather data. During this analysis, weather components such as temperature, humidity, precipitation and wind speed are taken into account. rainfall, and wind speeds are considered. From such an analysis, we can predict the behavior of the weather. one can be able to predict the behavior of the weather. The main objective of carrying out this project is to analyse the collected meteorological data in order to extract useful information. primary goal of conducting this project is to analyze the weather data collected in order to extract useful information from it. This will make weather forecasts easier, among other things. assist in weather predictions among other things. The project will make it easier for us to appreciate the usefulness of data analysis in addressing real-world problems such as weather changes. dealing with real-world issues such as weather changes.

2. Objective

Implementation of a simple weather data analysis system through the use of core software technologies such as Java, HTML and CSS. There are some key objectives of the current project:

Implementation of a simple weather data analysis system through usage of basic software technologies such as Java, HTML, and CSS. It allows you to learn how software and web development can be combined in a real project. learning how software and web development can be combined in a real project. Proper storage and organization of data regarding weather elements in the form of a Binary Search Tree (BST). It allows weather data management and quick search. makes weather data management and quick search possible. Search for weather elements such as temperature, humidity, precipitation and wind speed and their changes. Research of such weather elements as temperature, humidity, rainfall, and wind speed and their changes. Analysis of the obtained data and receipt of valuable and relevant information based on them. receiving of valuable and relevant information based on them. Using bar charts on the web page for visual demonstration of the analyzed data to make it easier for users to interpret. Usage of bar charts on the webpage for visual demonstration of analyzed data to make them easier to interpret by the users. Create a simple and practical web interface that is user-friendly and comfortable to use. Creating a simple and convenient web interface which will be user-friendly and comfortable to use. Improving our programming skills, implementing algorithms and data structures. Enhancement of our skills in programming, algorithms, and data structures implementation. Understanding of the application of data analysis in practice, particularly related to weather research and forecasting. application in practice, especially related to weather research and forecasting.

3. Technology stack

Technology Stack Used in Our Website

 HTML5: Used to create the basic structure of the website like header, search bar, weather section, graphics section and BST section. charts section, and BST section.

 CSS3 : Used to design and style the website. for designing and styling the website. It helps to create glass morphing effects, animated background, responsive layout, buttons, cards and overall attractive user interface. in creating glassmorphism effects, animated background, responsive layout, buttons, cards, and overall attractive UI.

 JavaScript (JS): Used to make the website functional and interactive. It manages search, location access, API data retrieval, UI updates and BST operation. handles search, location access, API data fetching, UI updates, and BST working.

 Open-Weather API : Used to retrieve live weather data such as temperature, humidity, wind speed, pressure and 7-day weather forecast. Open-Meteo APIUsed to fetch live weather data like temperature, humidity, wind speed, pressure, and 7-day weather forecast.

 Geocoding API : Used to convert user-entered city name to latitude and longitude for weather data retrieval. the city name entered by the user into latitude and longitude for weather data fetching.

 Geolocation API: Used to automatically detect the user's current location and display live weather data of that location. user’s current location automatically and show live weather data of that location.

 Graphic. Chart. Js: Used to create a graphical representation of weather data such as line charts and bar charts for better analysis. like Line Chart and Bar Chart for better analysis.

 Binary Search Tree (BST): Used as a Data Structure concept to store and sort temperature data from lowest to highest using in-order traversal. inorder traversal.

 Font Awesome: Used to add icons such as cloud, search, wind, humidity and location icons to improve the appearance of the website. for adding icons like cloud, search, wind, humidity, and location icons to improve the website appearance.

 Google Fonts (Outfit Fon): Used to improve the style of the text and give the website a modern and professional look. styling and make the website look modern and professional.

 Responsive Web Design : Used with CSS media queries to make the website work properly on mobile, tablet and desktop screens. tablet, and desktop screens.

4. SYSTEM DESIGN

4.1 SYSTEM OVERVIEW

The Weather Data Analysis (SkyCast) Website is a web-based system designed to provide real-time weather information in a simple and interactive manner. (SkyCast) is a web-based system designed to provide real-time weather information in a simple and interactive way. It allows users to search weather details about any city or use their current location to get live weather updates. of any city or use their current location to get live weather updates. The system displays temperature, humidity, wind speed, pressure and 7-day forecast using charts and graphical analysis. pressure, and 7-day forecast using charts and graphical analysis. It also uses Binary Search Tree (BST) to sort temperature data efficiently.

4.2 STRUCTURE OF THE SYSTEM

The system is mainly divided into three parts:

1. Frontend (User Interface) – Built using HTML, CSS and JavaScript to create the layout, design and user interaction of the website. CSS, and JavaScript to create the website layout, design, and user interaction.

2. API Integration – Open-Weather API and Geocoding API are used to retrieve live weather data and location coordinates. Open-Meteo API and Geocoding API are used to fetch live weather data and location coordinates.
3. Data Processing Section – Chart. js is used for graphing and BST is used to store and sort temperature data

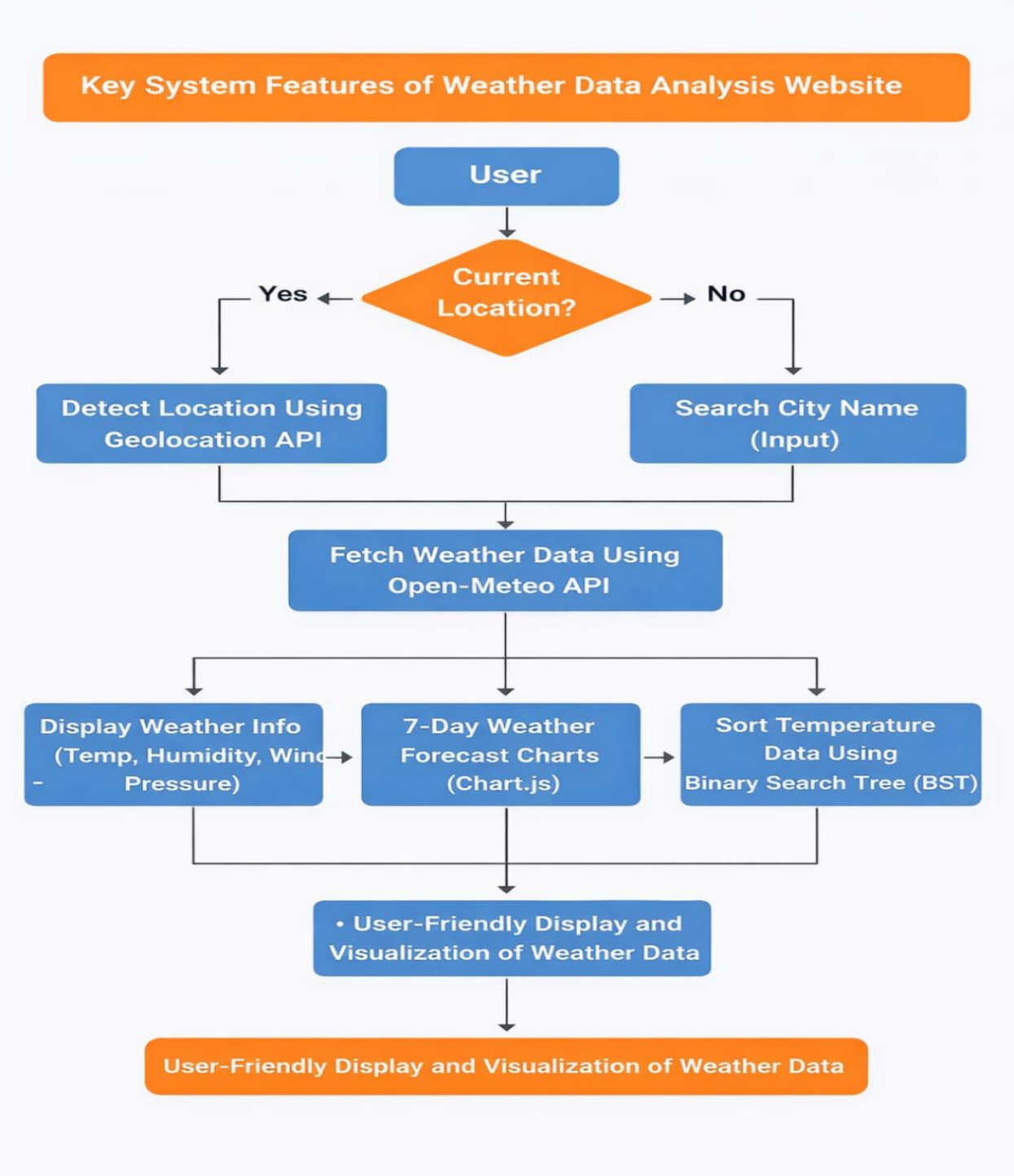
from lowest to highest. graphical representation, and BST is used to store and sort temperature data from lowest to highest.

4.3 SYSTEM DATA STRUCTURE DESIGN

The main data structure used in this project is **Binary Search Tree (BST)**. It stores temperature values along with dates. Each node contains date, temperature, left child, and right child. Smaller temperature values are stored on the left side, and larger values are stored on the right side. Inorder traversal is used to display temperatures in sorted order from lowest to highest.

4.4 HOW THE SYSTEM WORKS

When the website opens, it automatically detects the user's current location using Geolocation API. The user can also search any city manually. The system sends the request to the Geocoding API to get latitude and longitude, then fetches weather data from the Open-Meteo API. The fetched data is displayed on the screen, shown in charts using Chart.js, and inserted into BST for sorting. Finally, the sorted temperature data is displayed in the BST section for better analysis.


Key System Features of Weather Data Analysis Website
User
Current Location?
Yes
No
Detect Location Using Geolocation API
Search City Name (Input)
Fetch Weather Data Using Open-Meteo API
Display Weather Info (Temp, Humidity, Winc
Pressure)
7-Day Weather Forecast Charts (Chart.js)
Sort Temperature Data Using Binary Search Tree (BST)
• User-Friendly Display and Visualization of Weather Data
User-Friendly Display and Visualization of Weather Data

5. Codes

Index file

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SkyCast - Modern Weather Data Analysis</title>
    <!-- Fonts -->
    <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;700&display=swap" rel="stylesheet">
    <!-- FontAwesome for Icons -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
    <!-- Stylesheet -->
    <link rel="stylesheet" href="./css/style.css?v=2">
</head>
<body>
    <!-- Anim
    <div clas
        <div class="clouds"></div>
        <div class="particles" id="particles-container"></div>
    </div>

    <!-- Main Application Container -->
    <main class="app-container">
        <!-- Header & Search -->
        <header class="glass-panel">
            <h1><i class="fa-solid fa-cloud-sun-rain"></i> SkyCast</h1>
            <div class="search-container">
                <input type="text" id="city-input" placeholder="Search Indian states or cities...">
                <button id="search-btn"><i class="fa-solid fa-search"></i> Search</button>
                <button id="location-btn" title="Use My Location"><i class="fa-solid fa-location-crosshairs"></i></button>
            </div>
        </header>

        <!-- Current Weather Overview -->
        <section class="current-weather glass-panel" id="current-weather-panel">
            <div class="weather-main">
                <h2 id="location-name">Detecting Location...</h2>
                <div class="temp-display">
                    <i class="fa-solid fa-temperature-half temp-icon"></i>
```

A space-separated list of the classes of the element. Classes allows CSS and JavaScript to select and access specific elements via the class selectors or functions like the method `Document.getElementsByClassName()`.

MDN Reference

```html
                <span id="current-temp">--°C</span>
            </div>
            <p id="weather-description" class="weather-desc">Awaiting Data</p>
        </div>

        <div class="weather-details">
            <div class="detail-card">
                <i class="fa-solid fa-droplet"></i>
                <div>
                    <p class="label">Humidity</p>
                    <p class="value" id="current-humidity">--%</p>
                </div>
            </div>
            <div class="detail-card">
                <i class="fa-solid fa-wind"></i>
                <div>
                    <p class="label">Wind Speed</p>
                    <p class="value" id="current-wind">-- km/h</p>
                </div>
            </div>
            <!-- Adding extra data usually present in weather apps for visual completeness -->
            <div class="detail-card">
                <i class="fa-solid fa-gauge-high"></i>
                <div>
                    <p class="label">Pressure</p>
                    <p class="value" id="current-pressure">-- hPa</p>
                </div>
            </div>
        </div>
    </section>

    <!-- Charts Section -->
    <section class="charts-section">
```

```html
        <div class="chart-container glass-panel">
            <h3><i class="fa-solid fa-chart-line"></i> 7-Day Temperature Trend</h3>
            <canvas id="lineChart"></canvas>
        </div>
        <div class="chart-container glass-panel">
            <h3><i class="fa-solid fa-chart-bar"></i> Daily Comparison</h3>
            <canvas id="barChart"></canvas>
        </div>
    </section>

    <!-- Binary Search Tree Section -->
    <section class="bst-section glass-panel">
        <h3><i class="fa-solid fa-network-wired"></i> Sorted Temperature Records (BST Algorithm)</h3>
        <p class="bst-desc">The 7-day temperature data is inserted into a Binary Search Tree (BST). Below is the Inorder Traver
        <div class="bst-container" id="bst-container">
            <!-- Sorted cards will be appended here -->
            <div class="placeholder-text">Awaiting data...</div>
        </div>
    </section>
</main>

<!-- Scripts -->
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script src="./js/bst.js?v=2"></script>
<script src="./js/app.js?v=2"></script>
</body>
</html>
```

Style.CSS

```css
/* ==== CSS Tokens and Resets ==== */
:root {
    --bg-gradient-clear: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
    --bg-gradient-cloudy: linear-gradient(135deg, #757f9a 0%, #d7dde8 100%)
    --bg-gradient-rain: linear-gradient(135deg, #2c3e50 0%, #3498db 100%);
    --bg-gradient-night: linear-gradient(135deg, #141e30 0%, #243b55 100%);

    --glass-bg: rgba(255, 255, 255, 0.1);
    --glass-border: rgba(255, 255, 255, 0.2);
    --glass-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.3);

    --text-primary: #ffffff;
    --text-secondary: rgba(255, 255, 255, 0.8);
    --accent: #ffeb3b;
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Outfit', sans-serif;
}

body {
    background: var(--bg-gradient-clear);
    color: var(--text-primary);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    overflow-x: hidden;
    transition: background 1s ease;
}

/* ==== Animated Background ==== */
.sky-background {
    position: fixed;
    top: 0;
    left: 0;
```

```
    width: 100%;
    height: 100%;
    z-index: -1;
    overflow: hidden;
}

.clouds {
    position: absolute;
    width: 200%;
    height: 100%;
    background: transparent url('https://cdn.pixabay.com/photo/2016/02/10/20/00/clouds-1192668_1280.png') repeat-x;
    background-size: 50% 100%;
    opacity: 0.3;
    animation: moveClouds 100s linear infinite;
}

@keyframes moveClouds {
    0% {
        transform: translateX(0);
    }

    100% {
        transform: translateX(-50%);
    }
}

/* ==== Main Container ==== */
.app-container {
    max-width: 1200px;
    width: 100%;
    padding: 2rem;
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
    z-index: 10;
}
```

```
/* ==== Glassmorphism Utils ==== */
.glass-panel {
    background: var(--glass-bg);
    backdrop-filter: blur(12px);
    -webkit-backdrop-filter: blur(12px);
    border: 1px solid var(--glass-border);
    border-radius: 20px;
    padding: 1.5rem;
    box-shadow: var(--glass-shadow);
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.glass-panel:hover {
    transform: translateY(-2px);
    box-shadow: 0 12px 40px 0 rgba(0, 0, 0, 0.4);
}

/* ==== Header & Search ==== */
header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 1rem;
}

h1 {
    font-size: 2rem;
    font-weight: 700;
    letter-spacing: 1px;
    display: flex;
    align-items: center;
    gap: 10px;
}

h1 i {
    color: var(--accent);
}
```

```
.search-container {
    display: flex;
    gap: 0.5rem;
}

input[type="text"] {
    background: rgba(255, 255, 255, 0.2);
    border: 1px solid var(--glass-border);
    border-radius: 30px;
    padding: 0.8rem 1.5rem;
    color: white;
    font-size: 1rem;
    outline: none;
    width: 300px;
    transition: width 0.3s ease, background 0.3s;
}

input[type="text"]::placeholder {
    color: var(--text-secondary);
}

input[type="text"]:focus {
    background: rgba(255, 255, 255, 0.3);
}

button {
    background: rgba(255, 255, 255, 0.2);
    border: 1px solid var(--glass-border);
    border-radius: 30px;
    padding: 0.8rem 1.5rem;
    color: white;
    font-size: 1rem;
    cursor: pointer;
    transition: background 0.3s ease;
    display: flex;
    align-items: center;
    gap: 8px;
```

```
}

button:hover {
    background: rgba(255, 255, 255, 0.4);
}

#location-btn {
    padding: 0.8rem;
    border-radius: 50%;
}

/* ==== Current Weather Section ==== */
.current-weather {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 2rem;
}

.weather-main {
    flex: 1;
}

.weather-main h2 {
    font-size: 2.5rem;
    margin-bottom: 0.5rem;
}

.temp-display {
    display: flex;
    align-items: center;
    gap: 15px;
    font-size: 4rem;
    font-weight: 700;
}
```

```
.temp-icon {
    font-size: 3.5rem;
    color: var(--accent);
}

.weather-desc {
    font-size: 1.2rem;
    color: var(--text-secondary);
    text-transform: capitalize;
}

.weather-details {
    display: flex;
    gap: 1.5rem;
    flex-wrap: wrap;
}

.detail-card {
    background: rgba(0, 0, 0, 0.2);
    padding: 1rem 1.5rem;
    border-radius: 15px;
    display: flex;
    align-items: center;
    gap: 15px;
    min-width: 150px;
}

.detail-card i {
    font-size: 1.8rem;
    color: var(--accent);
}

.detail-card .label {
    font-size: 0.9rem;
    color: var(--text-secondary);
}

.detail-card .value {
```

```
.detail-card .value {
    font-size: 1.2rem;
    font-weight: 600;
}

/* ==== Charts Section ==== */
.charts-section {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
}

.chart-container h3 {
    margin-bottom: 1rem;
    font-weight: 600;
    font-size: 1.2rem;
    border-bottom: 1px solid var(--glass-border);
    padding-bottom: 0.5rem;
}

canvas {
    width: 100% !important;
    height: 300px !important;
}

/* ==== BST Section ==== */
.bst-section {
    margin-bottom: 2rem;
}

.bst-section h3 {
    margin-bottom: 0.5rem;
    border-bottom: 1px solid var(--glass-border);
    padding-bottom: 0.5rem;
}

.bst-desc {
```

```
        font-size: 0.9rem;
        color: var(--text-secondary);
        margin-bottom: 1rem;
    }

    .bst-container {
        display: flex;
        flex-wrap: wrap;
        gap: 1rem;
    }

    .bst-card {
        background: rgba(255, 255, 255, 0.15);
        border: 1px solid var(--glass-border);
        padding: 1rem;
        border-radius: 12px;
        text-align: center;
        min-width: 100px;
        transition: transform 0.2s;
    }

    .bst-card:hover {
        transform: scale(1.05);
        background: rgba(255, 255, 255, 0.25);
    }

    .bst-card .bst-temp {
        font-size: 1.5rem;
        font-weight: bold;
        color: var(--accent);
    }

    .bst-card .bst-date {
        font-size: 0.8rem;
        color: var(--text-primary);
    }
```

```

    /* ==== Responsive Design ==== */
    @media (max-width: 900px) {
        .charts-section {
            grid-template-columns: 1fr;
        }
    }

    @media (max-width: 600px) {
        input[type="text"] {
            width: 100%;
        }

        .current-weather {
            flex-direction: column;
            align-items: flex-start;
        }

        .weather-details {
            width: 100%;
            justify-content: space-between;
        }
    }
```

bst.js

```javascript
**
 * bst.js
 * Implementation of a Binary Search Tree to store and analyze temperature records.
 * For college micro project requirements.
 */

// Node class represents each individual node in the BST
class Node {
    constructor(date, temp) {
        this.date = date; // The date of the temperature
        this.temp = temp; // The temperature value (acts as the Key for BST)
        this.left = null; // Pointer to left child
        this.right = null; // Pointer to right child
    }
}

// Binary Search Tree class
class BinarySearchTree {
    constructor() {
        this.root = null; // Root node of the tree
    }

    /**
     * Insert a new temperature record into the BST
     * @param {string} date - Date string
     * @param {number} temp - Temperature
     */
    insert(date, temp) {
        const newNode = new Node(date, temp);

        if (this.root === null) {
            this.root = newNode;
        } else {
            this._insertNode(this.root, newNode);
        }
    }

    // Helper method for recursive insertion
```

```javascript
    _insertNode(node, newNode) {
        // If the new temp is less than the current node's temp, go left
        if (newNode.temp < node.temp) {
            if (node.left === null) {
                node.left = newNode;
            } else {
                this._insertNode(node.left, newNode);
            }
        }
        // Otherwise, go right
        else {
            if (node.right === null) {
                node.right = newNode;
            } else {
                this._insertNode(node.right, newNode);
            }
        }
    }

    /**
     * Inorder Traversal (Left, Root, Right)
     * For a BST, this traversal outputs the data in sorted (ascending) order.
     * @returns {Array} Array of temperature records sorted from coldest to hottest
     */
    inorder() {
        const result = [];
        this._inorderTraverse(this.root, result);
        return result;
    }

    // Helper method for recursive inorder traversal
    _inorderTraverse(node, result) {
        if (node !== null) {
            this._inorderTraverse(node.left, result);  // Visit Left
            result.push({ date: node.date, temp: node.temp }); // Visit Root
            this._inorderTraverse(node.right, result); // Visit Right
        }
```

```javascript
        }

        /**
         * Clear the tree (useful when searching a new city)
         */
        clear() {
            this.root = null;
        }
    }

    // Export to window object so app.js can access it
    window.BinarySearchTree = BinarySearchTree;
```

App.js

```javascript
/**
 * app.js
 * Handles UI interactions, Geolocation, API requests, and Chart rendering.
 */
 let lineChartInstance: any

// G app.js Handles UI interactions, Geolocation, API requests, and Chart rendering.
let lineChartInstance = null;
let barChartInstance = null;

// Initialize the BST
const temperatureTree = new window.BinarySearchTree();

// DOM Elements
const cityInput = document.getElementById('city-input');
const searchBtn = document.getElementById('search-btn');
const locationBtn = document.getElementById('location-btn');
const locNameDisplay = document.getElementById('location-name');
const curTempDisplay = document.getElementById('current-temp');
const curDescDisplay = document.getElementById('weather-description');
const curHumidDisplay = document.getElementById('current-humidity');
const curWindDisplay = document.getElementById('current-wind');
const curPressDisplay = document.getElementById('current-pressure');
const bstContainer = document.getElementById('bst-container');

// Mapping WMO Weather codes to descriptions (Open-Meteo format)
const weatherCodeMap = {
    0: "Clear sky",
    1: "Mainly clear", 2: "Partly cloudy", 3: "Overcast",
    45: "Fog", 48: "Depositing rime fog",
    51: "Light drizzle", 53: "Moderate drizzle", 55: "Dense drizzle",
    61: "Slight rain", 63: "Moderate rain", 65: "Heavy rain",
    71: "Slight snow", 73: "Moderate snow", 75: "Heavy snow",
    95: "Thunderstorm", 96: "Thunderstorm with slight hail", 99: "Thunderstorm with heavy hail"
};

// Application Initialization
window.addEventListener('DOMContentLoaded', () => {
    // Attempt to get user's live location on load
```

```javascript
    fetchWeatherByGeolocation();
});

// Event Listeners
searchBtn.addEventListener('click', handleSearch);
cityInput.addEventListener('keypress', (e) => {
    if (e.key === 'Enter') handleSearch();
});
locationBtn.addEventListener('click', fetchWeatherByGeolocation);

/* ==== Core Functions ==== */

function handleSearch() {
    const query = cityInput.value.trim();
    if (!query) return;

    // We update UI to indicate loading
    locNameDisplay.innerText = "Searching...";

    // Step 1: Geocoding API to get lat/lon from city name
    const geoUrl = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(query)}&count=1&language=en&format=js

    fetch(geoUrl)
        .then(res => res.json())
        .then(data => {
            if (!data.results || data.results.length === 0) {
                alert("Location not found. Try a valid Indian state or city.");
                locNameDisplay.innerText = "Search Failed";
                return;
            }
            const location = data.results[0];
            const name = `${location.name}, ${location.country}`;
            fetchWeatherData(location.latitude, location.longitude, name);
        })
        .catch(err => {
            console.error("Geocoding Error:", err);
            alert("Error fetching location data.");
```

```javascript
        });
}

function fetchWeatherByGeolocation() {
    locNameDisplay.innerText = "Detecting Location...";
    if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(
            (position) => {
                const lat = position.coords.latitude;
                const lon = position.coords.longitude;
                // Use reverse geocoding approximation or just show coordinates.
                // Open-Meteo doesn't have a direct free reverse geocoder in the main API,
                // so we will just label it "Your Location" or fetch nearest city if possible.
                fetchWeatherData(lat, lon, "Your Location");
            },
            (error) => {
                console.error("Geolocation Error:", error);
                alert("Geolocation access denied or failed. Please search manually.");
                locNameDisplay.innerText = "Search a city";
            }
        );
    } else {
        alert("Geolocation is not supported by your browser.");
        locNameDisplay.innerText = "Search a city";
    }
}

/**
 * Fetches current and 7-day weather data from Open-Meteo API
 * @param {number} lat - Latitude
 * @param {number} lon - Longitude
 * @param {string} displayName - Name to show in UI
 */
function fetchWeatherData(lat, lon, displayName) {
    // API URL fetching current weather + Daily bounds for 7 days
```

```
    const weatherUrl = `https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}&current_weather=true&hourly=relativ

    fetch(weatherUrl)
        .then(res => res.json())
        .then(data => {
            updateUI(data, displayName);
            updateCharts(data.daily);
            updateBST(data.daily);
        })
        .catch(err => {
            console.error("Weather API Error:", err);
            alert("Error fetching weather data.");
        });
}

/* ==== UI Updates ==== */

function updateUI(data, displayName) {
    const current = data.current_weather;
    const hourly = data.hourly;

    // We use the first hourly data for humidity/pressure as a proxy for current
    const currentHumidity = hourly.relativehumidity_2m[0];
    const currentPressure = hourly.surface_pressure[0];

    locNameDisplay.innerText = displayName;
    curTempDisplay.innerText = `${current.temperature}°C`;
    curWindDisplay.innerText = `${current.windspeed} km/h`;

    // Humidity & Pressure
    curHumidDisplay.innerText = `${currentHumidity}%`;
    curPressDisplay.innerText = `${currentPressure} hPa`;

    // Map Weather Code to Description
    const desc = weatherCodeMap[current.weathercode] || "Unknown Conditions";
    curDescDisplay.innerText = desc;
```

```

        // Update Background dynamically based on weather code
        updateBackground(current.weathercode);
    }

    function updateBackground(code) {
        const body = document.body;
        // Simple logic:
        // 0-2: Clear -> var(--bg-gradient-clear)
        // 3, 45-48: Cloudy -> var(--bg-gradient-cloudy)
        // 51-65: Rain -> var(--bg-gradient-rain)
        if (code <= 2) {
            body.style.background = "var(--bg-gradient-clear)";
        } else if (code <= 48) {
            body.style.background = "var(--bg-gradient-cloudy)";
        } else {
            body.style.background = "var(--bg-gradient-rain)";
        }
    }

    /* ==== Chart.js Integration ==== */

    function updateCharts(dailyData) {
        const dates = dailyData.time;
        const maxTemps = dailyData.temperature_2m_max;
        const minTemps = dailyData.temperature_2m_min;

        // Calculate Average
        const avgTemps = maxTemps.map((max, i) => (max + minTemps[i]) / 2);

        // Destroy existing charts to prevent memory leaks and overlay bugs
        if (lineChartInstance) lineChartInstance.destroy();
        if (barChartInstance) barChartInstance.destroy();

        // Setup Line Chart (Trend)
        const ctxLine = document.getElementById('lineChart').getContext('2d');
        lineChartInstance = new Chart(ctxLine, {
```

```
            type: 'line',
            data: {
                labels: dates,
                datasets: [{
                    label: 'Average Temp (°C)',
                    data: avgTemps,
                    borderColor: '#ffeb3b',
                    backgroundColor: 'rgba(255, 235, 59, 0.2)',
                    borderWidth: 3,
                    tension: 0.4, // Smooth curve
                    fill: true
                }]
            },
            options: getChartOptions()
        });

        // Setup Bar Chart (Min vs Max Comparison)
        const ctxBar = document.getElementById('barChart').getContext('2d');
        barChartInstance = new Chart(ctxBar, {
            type: 'bar',
            data: {
                labels: dates,
                datasets: [
                    {
                        label: 'Max Temp (°C)',
                        data: maxTemps,
                        backgroundColor: '#ff5252'
                    },
                    {
                        label: 'Min Temp (°C)',
                        data: minTemps,
                        backgroundColor: '#448aff'
                    }
                ]
            },
```

```
        },
        options: getChartOptions()
    });
}

function getChartOptions() {
    return {
        responsive: true,
        plugins: {
            legend: {
                labels: { color: 'white' }
            }
        },
        scales: {
            x: { ticks: { color: 'rgba(255,255,255,0.8)' }, grid: { color: 'rgba(255,255,255,0.1)' } },
            y: { ticks: { color: 'white' }, grid: { color: 'rgba(255,255,255,0.1)' } }
        }
    };
}

/* ==== BST Execution ==== */

function updateBST(dailyData) {
    // 1. Clear previous data
    temperatureTree.clear();
    bstContainer.innerHTML = '';

    const dates = dailyData.time;
    const maxTemps = dailyData.temperature_2m_max;

    // 2. Insert records into BST
    for (let i = 0; i < dates.length; i++) {
        // We use max temperature for simplicity
        temperatureTree.insert(dates[i], maxTemps[i]);
    }
```

```
    // 3. Traverse to get sorted order
    const sortedRecords = temperatureTree.inorder();

    // 4. Render to UI
    sortedRecords.forEach(record => {
        const card = document.createElement('div');
        card.className = 'bst-card';
        card.innerHTML = `
            <div class="bst-temp">${record.temp}ºC</div>
            <div class="bst-date">${formatDate(record.date)}</div>
        `;
        bstContainer.appendChild(card);
    });
}

// Utility function to format dates to "Mon, 12"
function formatDate(dateString) {
    const d = new Date(dateString);
    return d.toLocaleDateString('en-US', { weekday: 'short', day: 'numeric' });
}
```

6. ANALYSIS OF RESULTS

6.1 RESULTS THAT WORK

The system successfully:

- Fetches live weather data using API
- Displays temperature, humidity, wind speed, and pressure
- Shows 7-day weather forecast using charts
- Sorts temperature data using Binary Search Tree (BST)
- Provides smooth and responsive user interface
- Detects current location using Geolocation API

6.1 PERFORMANCES ANALYSIS

Table 1 : Table for performance analysis

Operation	Complexity
API Data Fetching	Real-time
BST Insert	O(log n)
BST Traversal	O(n)
Chart Rendering	Real-time
UI Update	Fast Response

6.2 BENEFITS

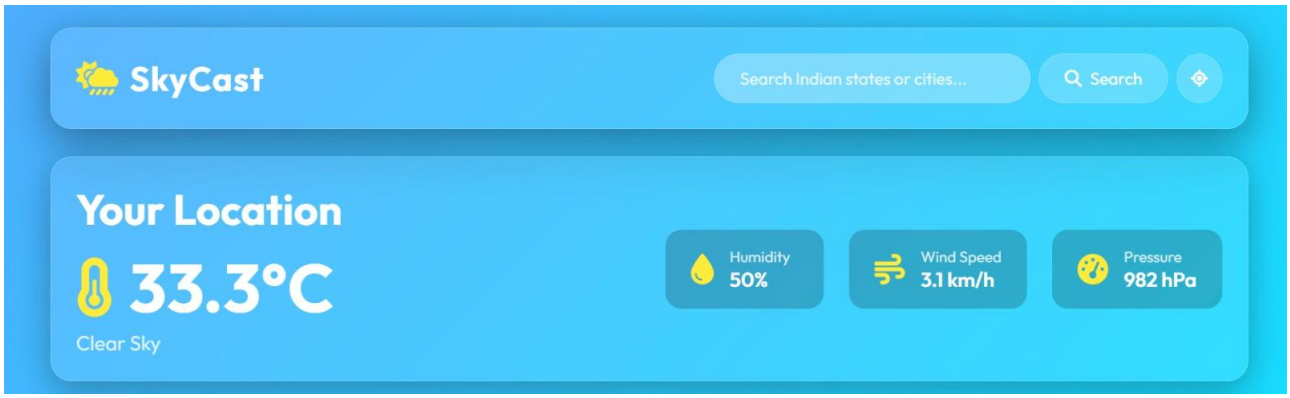
• Real-time weather information • Easy to understand graphical analysis • Efficient temperature sorting with BST • Attractive and interactive user interface • Useful for learning web development and DSA concepts Easy-to-understand graphical analysis • Efficient temperature sorting using BST • Attractive and interactive user interface • Useful for learning Web Development and DSA concepts

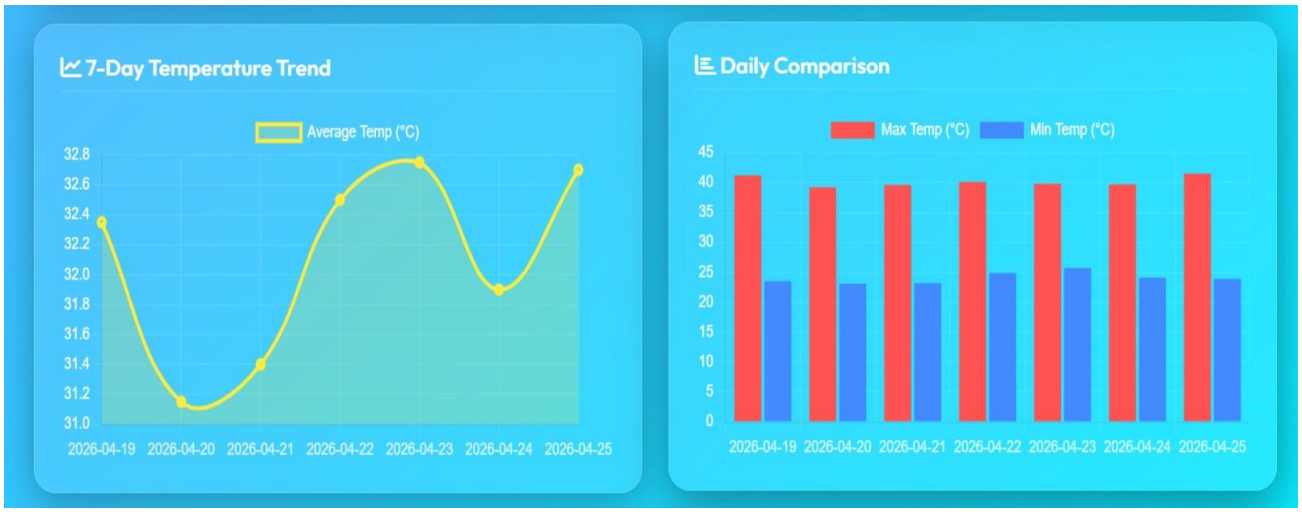
6.3 LIMITATIONS

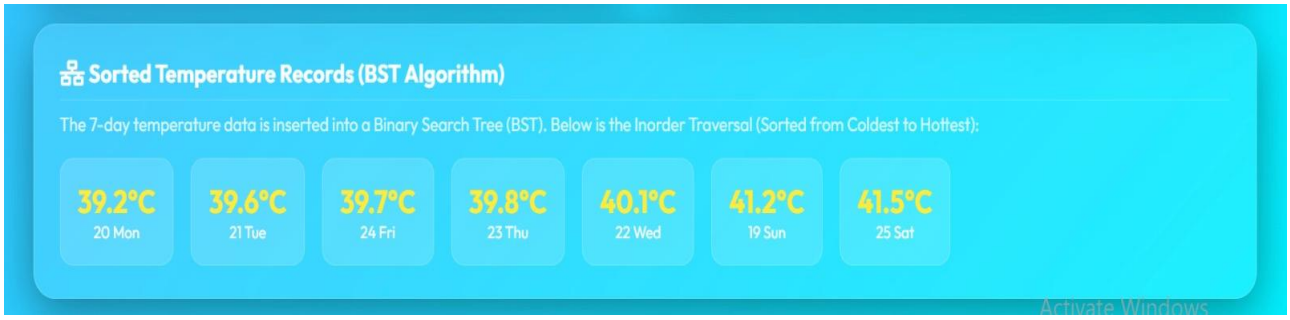
- Requires internet connection to fetch data
- Depends on browser location permission
- No database storage for previous searches

- API accuracy depends on external weather service

7. OUTPUT







8. CONCLUSION

Weather Data Analysis Website (SkyCast) is a modern web application that provides live weather updates in a simple and attractive way. Weather Data Analysis Website (SkyCast) is a modern web application that provides live weather updates in a simple and attractive way. The Weather Data Analysis Website (SkyCast) is a modern web application that provides live weather updates in a simple and attractive way. It displays temperature, humidity, wind speed, pressure and 7-day forecast using tables and graphs. displays temperature, humidity, wind speed, pressure and 7-day forecast using tables and graphs. shows temperature, humidity, wind speed, pressure, and 7-day forecast using charts and graphs. The project also uses Binary Search Tree (BST) to sort temperature data from lowest to highest, which adds a concept of Data Structure to the website. concept au site Web. to the website. Use of HTML, CSS, JavaScript, API and graphic integration. graphics integration. Use of HTML, CSS, JavaScript, API and Chart integration. Using HTML, CSS, JavaScript, API integration, and Chart. js, the website becomes responsive, user-friendly and useful for real-time weather analysis. user-friendly and useful for real-time weather analysis. user-friendly, and useful for real-time weather analysis. Overall, this project effectively combines web development and data analysis. analysis efficiently. in an effective way.

9. FUTURE SCOPE

In the future, we may see this site expand with the addition of more weather details, including air quality, UV index, and weather alerts for rain or storms. In the future, we may see this site expand with the addition of more weather details, including air quality, UV index, and weather alerts for rain or storms. future we may see this site grow with the addition of more weather details which include air quality, UV index and weather alerts for rain or storms. We can Also add features to connect users who will, in turn, save their favorite locations. functionality for logging in users who will, in turn, save their favorite locations. we can add functionality for logging in users who will, in turn, save their favorite locations. may add in user login features which in turn will save their favourite locations. We also believe that the addition of more advanced maps and support for mobile apps will make the project a better and more user-friendly tool for real-time weather analysis. applications will make the project a better and more user-friendly tool for real-time weather analysis. I think adding more advanced maps and supporting mobile apps will make the project a better and more user-friendly tool for real-time weather analysis. that adding more advanced charts and mobile app support will make the project a better , more user friendly tool for real time weather analysis.

10. REFERENCES

1. Online articles and tutorials on weather data analysis.

Link : https://youtu.be/pFvWwFua6mw?si=TYqg76dcf_43iyJa.

2. Academic notes and materials related to web technologies , data structure and project development.

3. Open-Meteo weather API documentation for live weather data and forecasting .

4. Open-Meteo Geocoding API for converting city names into latitudes and longitude.

5. chart.js official documentation for creating line charts and bar charts.

PLAGIARISM CHECK CERTIFICATE

This is to certify that We, students of B. Tech. in **Centre for Computer Science and Technology,** have checked our complete report entitled **"Weather Data Analysis "** for similarity/plagiarism using the "Turnitin" software available in the institute.

This is to certify that the similarity in our report is found to be ……% with AI writing .....%

which is within the specified limit (20%).

The full plagiarism report, along with the summary, is enclosed.

Prerna Mishra
(BTTC25O1051)
Priyal Choudhary
(BTTC25O0153)

Checked & Approved By:

Mrs. Aditi Samadhiya
Assistant Professor
Centre for Computer Science and Technology
MITS Deemed University, Gwalior